

Excelでお仕事!

「ダウンロード」のメニュー

[そのまま使えるExcelツール集](#)

[組み込み用ExcelVBAモジュール集](#)

[「プログレスバー」のクラス](#)

[「日付入力」のクラス](#)

[「カレンダー入力」用フォーム](#)

[カレンダーテーブル取得クラス](#)

[「ふりがな自動入力」のサブクラス](#)

[実行時エラーのログ出力処理](#)

[メール送信\(BASP21利用\)](#)

[メール送信\(CDO利用\)](#)

[ファイル圧縮\(UNLHA32利用\)](#)

[ファイル圧縮\(7-ZIP32、7-ZIP64利用\)](#)

[自動採番機能\(オートナンバー\)](#)

[カンマ数不定のCSVファイル読み込み](#)

[自由設定のCSVファイル出力](#)

[フォルダの参照](#)

[Excelをダウンロードコンポーネントとして使う。](#)

[モジュール自動入れ替え機能](#)

[パスワード生成クラス](#)

[VisualBasic開発系のフリーソフト](#)

[一般用フリーソフト\(Excel不要\)](#)

現在表示中のメニューには▶が表示されます。
●の分類名をクリックすると内容が展開されて表示されます。

その他ご案内

↑ 広告です(表示されない時はこちら) ↑

他ジャンルは左の「カテゴリー」から！

[「トップページ」はこちら](#)

[「サイトマップ」はこちら](#)

[「さくいん」も用意してあります。](#)

[「更新履歴」はこちら](#) 

[「リンクページ」はこちら](#)

[「管理人のプロフィール」はこちら](#)

[管理人への「意見・質問」はこちら](#)

↓ 探し物が見つからない方はこちら ↓

Google

GoogleでWeb検索

Last Update : 11/25/2019 21:10:13

組み込み用モジュール集

カンマ数不定のCSVファイル読み込み

レコードによってカンマ数が一定していないCSV形式テキストファイルの読み込み処理です。ダブルクォーテーションに囲われた中での改行(項目内改行)についても対応しています。

このページの上半分のサンプルは古い形式のCSVを読む仕様です。

「古い形式」というのは、「CSV形式テキストデータの書き出し(VBA応用)」の一番上のサンプルで書き出したような形式を指しています
これに対して、そもそもExcelの一般作業で「名前を付けて保存」を選んで、ファイルの種類を「CSV(カンマ区切り)(*.csv)」や「CSV UTF-8(カンマ区切り)(*.csv)」で保存させたCSV形式ファイルをExcelシートに直接開くのではなく、内部処理でメモリ上で展開できるようにするサンプルを作成して、このページの後半に公開しました。

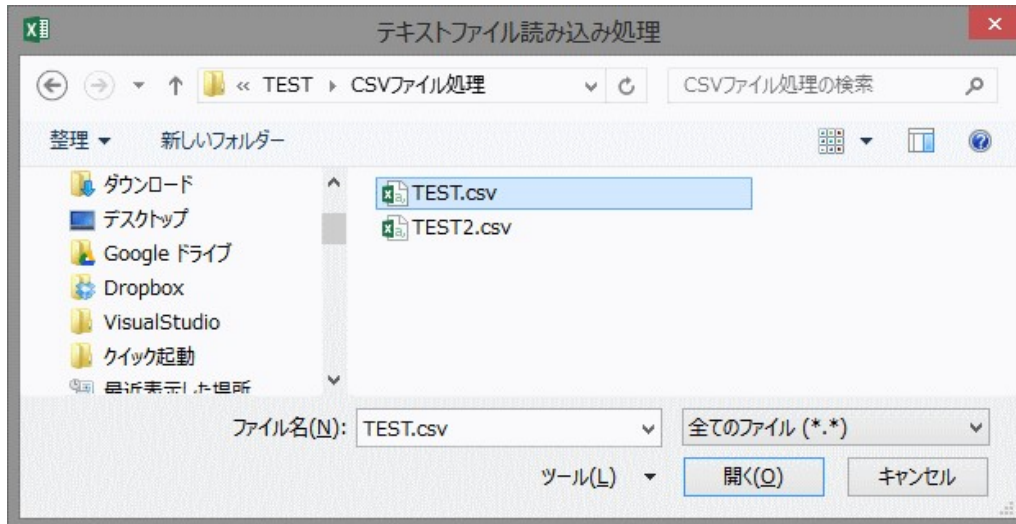
※Excel 2016以降では「カンマ区切り」が「コンマ区切り」という名称に変わっていますが、当サイトでは「カンマ区切り」に統一しています。

組み込み用モジュールと、サンプルを用意しました。



(この画像をクリックすると、ダウンロードができます。)

サンプルは起動させると、「ファイルを開く」のダイアログが表示されます。



CSV形式テキストファイルを指定して「開く」をクリックすると読み込みが行われ、シートの2行目から表示されます。

組み込みモジュールは「modGetCSVRec. bas」を使います。この中の「FP_GET_CSV_REC」というFunctionプロシージャにOpenしたファイルナンバーを引数として呼び出すと、CSV項目が配列となって戻されます。

```

' *****
'   CSV形式ファイル読み込み処理(カンマ数不定処理)
'
'   作成者:井上治   URL:http://www.ne.jp/asahi/excel/inoue/ [Excelでお仕事!]
' *****
Option Explicit
Const cnsTITLE = "テキストファイル読み込み処理"
Const cnsFILTER = "全てのファイル (*.*) *.*)"
Const cnsStartRow = 2

' *****
'   CSV形式テキストファイル(不定カラム)読み込みサンプル
' *****
Sub READ_TextFile()
    Dim xlAPP As Application      ' Applicationオブジェクト
    Dim intFF As Integer          ' FreeFile値
    Dim strFILENAME As String     ' OPENするファイル名(フルパス)
    Dim lX1 As Long              ' CSV項目カラムINDEX
    Dim lY0 As Long              ' 収容するセルの行
    Dim lngREC As Long           ' レコード件数カウンタ
    Dim vntREC As Variant         ' レコード内容(配列)

    ' Applicationオブジェクト取得
    Set xlAPP = Application
    ' 「ファイルを開く」のフォームでファイル名の指定を受ける
    xlAPP.StatusBar = "読み込むファイル名を指定して下さい。"
    strFILENAME = xlAPP.GetOpenFilename(FileFilter:=cnsFILTER, _
        Title:=cnsTITLE)
    ' キャンセルされた場合は以降の処理は行なわない
    If StrConv(strFILENAME, vbUpperCase) = "FALSE" Then Exit Sub

    ' FreeFile値の取得(以降この値で入出力する)
    intFF = FreeFile
    ' 指定ファイルをOPEN(入力モード)
    Open strFILENAME For Input As #intFF
    lY0 = cnsStartRow - 1
    ' ファイルのEOF(End of File)まで繰り返す
    Do Until EOF(intFF)
        ' レコード件数カウンタの加算
        lngREC = lngREC + 1
        xlAPP.StatusBar = "読み込み中です. . . . (" & lngREC & "レコード目)"

        ' 行単位にレコードを読み込む(共通処理)

```

```

vntREC = modGetCSVRec.FP_GET_CSV_REC(intFF)
IX1 = UBound(vntREC) + 1

' 行を加算しレコード内容を表示(先頭は2行目)
GY0 = GY0 + 1
Range(Cells(GY0, 1), Cells(GY0, IX1)).Value = vntREC ' 配列渡し
Loop
' 指定ファイルをCLOSE
Close #intFF
xlAPP.StatusBar = False
' 終了の表示
MsgBox "ファイル読み込みが完了しました。" & vbCrLf & _
"レコード件数=" & lngREC & "件", vbInformation, cnsTITLE
Set xlAPP = Nothing
End Sub

' -----<< End of Source >>-----

```

- ①「FP_GET_CSV_REC」をOpenしたファイルナンバーを引数として呼び出します。戻り値の受け取り変数「vntREC」はVariant型変数として下さい。
- ②セルの列は「1」起算、テーブルで返される変数は「0」起算のため、配列要素数の受け取りには「1」を加算します。
- ③セルに配列の順を崩さずに左から並べて収容するには、このように配列のまま渡すことができます。1要素ずつであれば、「vntREC(n)」を参照して渡します。(nの範囲はLBound(vntREC)からUBound(vntREC)までです。

ここで呼ばれている「FP_GET_CSV_REC」は組み込みやうに別モジュールにしています。
(但し、最後の説明までお読みの上でのご利用をお願いします。)

```

' *****
'   CSV形式ファイル読み込み処理(カンマ数不定処理)   ※2008/02/03修正
'
'   作成者:井上治   URL:http://www.ne.jp/asahi/excel/inoue/ [Excelでお仕事!]
' *****
Option Explicit
Const cnsDQ = """"
Const cnsDQCM = """"
Const cnsCOM = " , "
Const cnsBLNK = ""
Const cnsPROD = "."

' *****
'   CSV形式のレコードの受け取り   ※現時点ではエラー処理なし
'   (引数はファイルNo, 戻り値はCSVレコード内容の配列)
' *****
Public Function FP_GET_CSV_REC(intFF As Integer) As Variant
Dim strREC As String ' レコード内容(連結後)
Dim strREC2 As String ' レコード内容(連結前)
Dim strTEXT As String ' 1項目の内容
Dim strTEXT2 As String ' Work
Dim POS As Long ' 項目先頭から
Dim POS2 As Long ' 項目間のカンマ位置
Dim X() As Variant ' レコード内容WORK
Dim IX As Long ' 項目INDEX
Dim lngLEN As Long ' レコード長
Dim swTRUE As Boolean ' レコード連結判断スイッチ
Dim swDQ As Byte ' タブルクォーテーションスイッチ
Dim lngERR As Long ' エラーコード
Dim dteDate As Date ' 日付試験用Work

strREC = ""
' 項目途中改行対応のため、行末判定は独自に行なう
Do Until swTRUE = True

    swTRUE = True
    ' レコードの読み込み
    Line Input #intFF, strREC2
    ' 分断レコード対応のため文字列を接合する
    strREC = strREC & strREC2
    lngLEN = Len(strREC)

    ' 配列を初期化
    IX = -1
    ReDim X(0)
    ' レコード内容を1文字ずつ判定する
    POS = 1
    Do While POS <= lngLEN

        POS2 = POS + 1
        swDQ = 0
        If Mid$(strREC, POS, 1) = cnsDQ Then
            swDQ = 1
            ' 先頭がタブルクォーテーションの場合、項目末のタブルクォーテーションを探す
            Do While POS2 < lngLEN
                If Mid$(strREC, POS2, 2) = cnsDQCM Then Exit Do
                POS2 = POS2 + 1
            Loop

```

```

If POS2 >= lngLEN Then
    ' 行末に達した場合は正しい文字列か判定する
    If Right$(strREC, 1) = cnsDQ Then
        strTEXT = Trim$(Mid$(strREC, POS + 1, lngLEN - POS - 1))
    Else
        ' 不揃いの場合は次レコードを読み込むように指示する
        strTEXT = cnsBLNK
        swTRUE = False
    End If
ElseIf POS2 > (POS + 1) Then
    ' 両端のダブルクォーテーションを外す
    strTEXT = Trim$(Mid$(strREC, POS + 1, POS2 - POS - 1))
Else
    strTEXT = cnsBLNK
End If
POS2 = POS2 + 1
ElseIf Mid$(strREC, POS, 1) = cnsCOM Then
    ' カマのみの場合はEmptyをセットさせる
    strTEXT = ""
    POS2 = POS2 - 1
Else
    ' 先頭がダブルクォーテーションでない場合は単純にカンマを探す
    Do While POS2 <= lngLEN
        If Mid$(strREC, POS2, 1) = cnsCOM Then Exit Do
        POS2 = POS2 + 1
    Loop
    If POS2 > POS Then
        strTEXT = Trim$(Mid$(strREC, POS, POS2 - POS))
    Else
        strTEXT = cnsBLNK
    End If
End If

' テーブル要素数を追加して内容をセット
IX = IX + 1
ReDim Preserve X(IX)
strTEXT2 = StrConv(strTEXT, vbUpperCase)
If ((IsNumeric(strTEXT) = True) And (swDQ <> 1)) Then
    ' 数値でダブルクォーテーションで囲われていない
    If InStr(1, strTEXT, cnsPROD, vbTextCompare) <> 0 Then
        X(IX) = CDbI(strTEXT) ' 実数は浮動小数点型
    Else
        X(IX) = CCur(strTEXT) ' 整数は通貨型
    End If
ElseIf IsDate(strTEXT) Then
    X(IX) = CDate(strTEXT) ' 日付型
    On Error Resume Next
    dteDate = X(IX)
    lngERR = Err.Number
    On Error GoTo 0
    If lngERR <> 0 Then
        ' 日付エラー!
        X(IX) = strTEXT ' 文字列型に変更
    ElseIf dteDate < #1/1/1900# Then
        ' 日付範囲外
        X(IX) = strTEXT ' 文字列型に変更
    Else
        X(IX) = dteDate
    End If
ElseIf ((strTEXT2 = "TRUE") Or (strTEXT2 = "FALSE")) Then
    X(IX) = CBool(strTEXT) ' Boolean型
ElseIf strTEXT <> cnsBLNK Then
    X(IX) = strTEXT ' 文字列型
Else
    ' ブランクの場合は初期化 (Empty)
    X(IX) = Empty
End If

POS = POS2 + 1
Loop

' EOFの場合は無条件に終了とする
If EOF(intFF) = True Then swTRUE = True

Loop

' 配列を戻り値にセット
FP_GET_CSV_REC = X
End Function

' -----<< End of Source >>-----

```

組み込み用「ツール」としてありますが、「FP_GET_CSV_REC」内ではエラー処理を行なっていません。実行時エラーに関する処理が必要であれば組み込んで下さい。

また、CSV形式テキストファイルの「仕様」について所望するものと違う場合がありますのでご注意下さい。



←ReadCSVFile2.zip
(20KB)

FS0 (FileSystemObject) 版はこちらです。

上記と同様の機能をFS0 (FileSystemObject) で記述したものです。

```

' *****
'   CSV形式ファイル読み込み処理(カンマ数不定処理)   ※FS0用(2008/02/03修正)
'
'   作成者:井上治   URL:http://www.ne.jp/asahi/excel/inoue/ [Excelでお仕事!]
' *****
' [参照設定]
'   ・ Microsoft Scripting Runtime
' *****
Option Explicit
Const cnsDQ = """"
Const cnsDQCM = """, "
Const cnsCOM = " , "
Const cnsBLNK = ""
Const cnsPROD = "."

' *****
'   CSV形式の1レコードの受け取り   ※現時点ではエラー処理なし
'   (引数はTextStreamオブジェクト, 戻り値はCSVレコード内容の1次配列)
' *****
Public Function FP_GET_CSV_REC2(objTS As TextStream) As Variant
    Dim strREC As String      ' レコード内容(連結後)
    Dim strREC2 As String     ' レコード内容(連結前)
    Dim strTEXT As String     ' 1項目の内容
    Dim strTEXT2 As String    ' Work
    Dim POS As Long           ' 項目先頭コラム
    Dim POS2 As Long          ' 項目間のカンマ位置
    Dim X() As Variant        ' レコード内容WORK
    Dim IX As Long            ' 項目 INDEX
    Dim lngLEN As Long        ' レコード長
    Dim swTRUE As Boolean     ' レコード連結判断スイッチ
    Dim swDQ As Byte          ' ダブルクォーテーションスイッチ
    Dim lngERR As Long        ' エラーコード
    Dim dteDate As Date       ' 日付試験用Work

    strREC = ""
    ' 項目途中改行対応のため、行末判定は独自に行なう
    Do Until swTRUE = True

        swTRUE = True
        ' レコードの読み込み(TextStream)
        strREC2 = objTS.ReadLine
        ' 分断レコード対応のため文字列を接合する
        strREC = strREC & strREC2
        lngLEN = Len(strREC)

        ' 配列を初期化
        IX = -1
        ReDim X(0)
        ' レコード内容を1文字ずつ判定する
        POS = 1
        Do While POS <= lngLEN

            POS2 = POS + 1
            swDQ = 0
            If Mid$(strREC, POS, 1) = cnsDQ Then
                swDQ = 1
                ' 先頭がダブルクォーテーションの場合、項目末の同文字を探す
                Do While POS2 < lngLEN
                    If Mid$(strREC, POS2, 2) = cnsDQCM Then Exit Do
                    POS2 = POS2 + 1
                Loop
            If POS2 >= lngLEN Then
                ' 行末に達した場合は正しい文字列か判定する
                If Right$(strREC, 1) = cnsDQ Then
                    strTEXT = Trim$(Mid$(strREC, POS + 1, lngLEN - POS - 1))
                Else
                    ' 不揃いの場合は次レコードを読み込むように指示する
                    strTEXT = cnsBLNK
                    swTRUE = False
                End If
            ElseIf POS2 > (POS + 1) Then
                ' 両端のダブルクォーテーションを外す
                strTEXT = Trim$(Mid$(strREC, POS + 1, POS2 - POS - 1))
            Else
                strTEXT = cnsBLNK
            End If
            POS2 = POS2 + 1
            ElseIf Mid$(strREC, POS, 1) = cnsCOM Then

```

```

' カンマのみの場合はEmptyをセットさせる
strTEXT = ""
POS2 = POS2 - 1
Else
' 先頭がダブルクォーテーションでない場合は単純にカンマを探す
Do While POS2 <= lngLEN
If Mid$(strREC, POS2, 1) = cnsCOM Then Exit Do
POS2 = POS2 + 1
Loop
If POS2 > POS Then
strTEXT = Trim$(Mid$(strREC, POS, POS2 - POS))
Else
strTEXT = cnsBLNK
End If
End If

' テーブル要素数を追加して内容をセット
IX = IX + 1
ReDim Preserve X(IX)
strTEXT2 = StrConv(strTEXT, vbUpperCase)
If ((IsNumeric(strTEXT) = True) And (swDQ <> 1)) Then
' 数値でダブルクォーテーションで囲われていない
If InStr(1, strTEXT, cnsPROD, vbTextCompare) <> 0 Then
X(IX) = CDbI(strTEXT) ' 実数は浮動小数点型
Else
X(IX) = CCur(strTEXT) ' 整数は通貨型
End If
ElseIf IsDate(strTEXT) Then
X(IX) = CDate(strTEXT) ' 日付型
On Error Resume Next
dteDate = X(IX)
lngERR = Err.Number
On Error GoTo 0
If lngERR <> 0 Then
' 日付エラー!
X(IX) = strTEXT ' 文字列型に変更
ElseIf dteDate < #1/1/1900# Then
' 日付範囲外
X(IX) = strTEXT ' 文字列型に変更
Else
X(IX) = dteDate
End If
ElseIf ((strTEXT2 = "TRUE") Or (strTEXT2 = "FALSE")) Then
X(IX) = CBool(strTEXT) ' Boolean型
ElseIf strTEXT <> cnsBLNK Then
X(IX) = strTEXT ' 文字列型
Else
' ブランクの場合は初期化 (Empty)
X(IX) = Empty
End If

POS = POS2 + 1
Loop

' EOFの場合は無条件に終了とする
If objTS.AtEndOfStream Then swTRUE = True

Loop

' 配列を戻り値にセット
FP_GET_CSV_REC2 = X
End Function

' -----<< End of Source >>-----

```

こちらは、textStream自体を引数にしています。



←ReadCSVFile3.zip
(18KB)

IsNumeric関数についてご注意下さい。

本機能では、数字文字列のチェックにIsNumeric関数を用いていますが、このチェックは場合によっては誤動作することがあります。これはIsNumeric関数が16進数表現の文字を含む数字文字列を「数値」と判断することがあるからです。これを避けるためには、IsNumeric関数ではなく、WorksheetFunctionのIsNumber関数を使うべきなのですが、このサンプルはExcelVBAだけでなく旧VisualBasicユーザーも参考になれるため、あえてIsNumeric関数で押し通しています。数値検査のための専用関数を作ることも可能なわけですが、CSV形式ファイルのやりとりを主眼とする場所での説明には不向きなく、これも避けているわけですから、この点はご了承下さい。

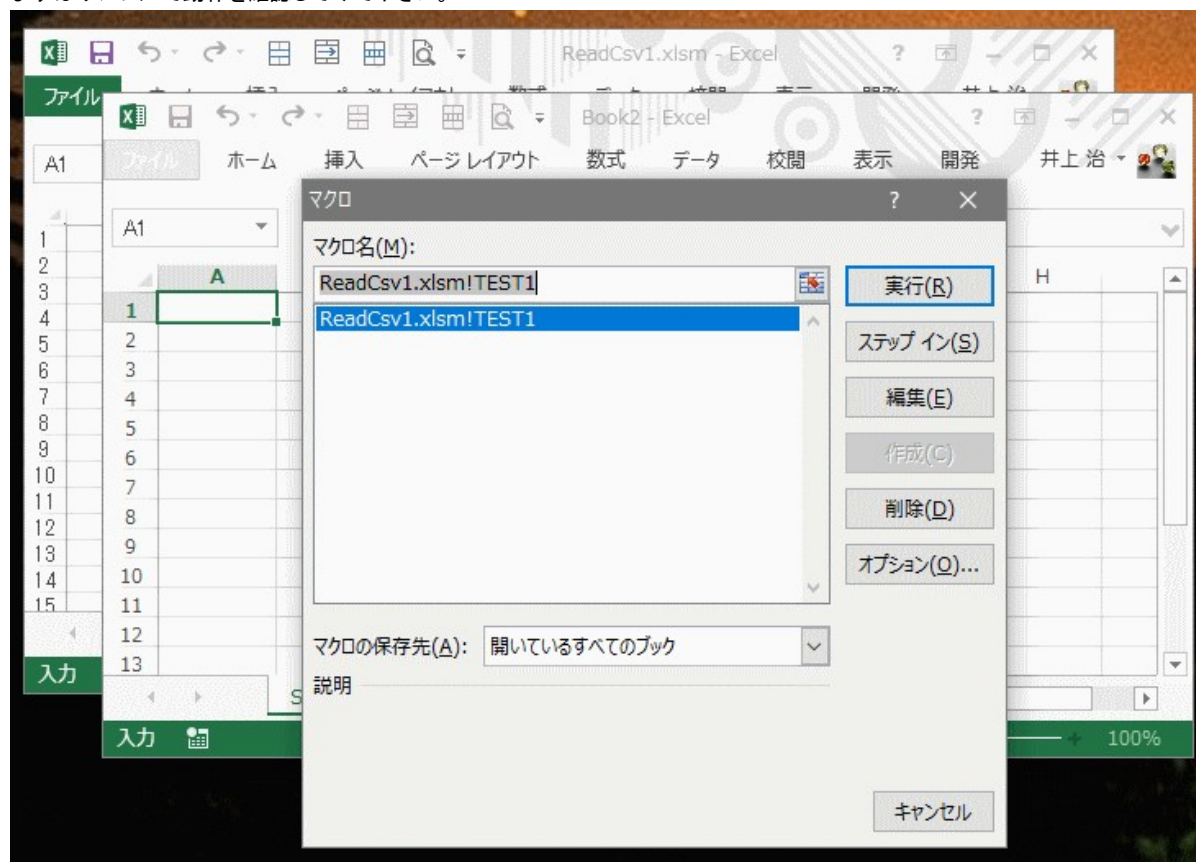
Excelで「名前を付けて保存」させたCSV形式ファイルの読み出し

ここからはこれまでのサンプルとは異なり「Excelで名前を付けて保存させたCSV形式ファイル」をいきなりワークシートに展

開するのではなく、VBAのコードで処理します。これは最近ではWebシステムとのやりとりでこのような形式が増えてきています。

「Excelで名前を付けて保存させたCSV形式ファイル」は後で説明しますが、「セル内で改行」があったり、「セル内で半角カンマ」が使われたりするため、CSV形式ファイルは単純に「改行で分解させて1件として処理」とか「カンマで分解させて項目ごとに分ける」ができないのです。

まずはサンプルで動作を確認してみてください。



(この画像をクリックすると、ダウンロードができます。)

ダウンロードしたZIPファイルを解凍すると、以下のファイルが出現します。

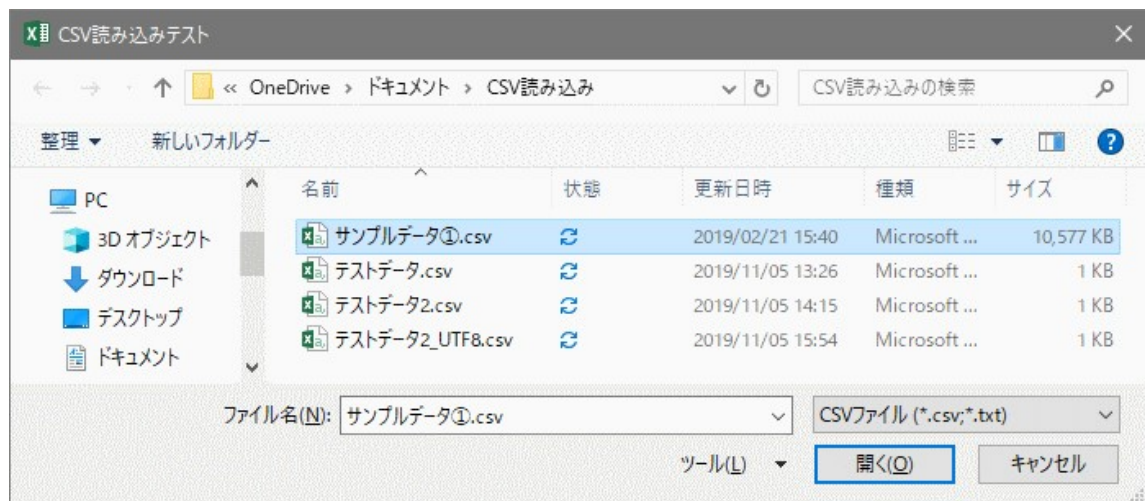
ファイル名	内容等
ReadCsvFile4SJIS.xlsm	CSV形式ファイル読み出し機能サンプル(シフトJIS版) ※マクロ有効ブック サンプルの起動プロシージャは「TEST1」です。(起動ボタン等はありません)
ReadCsvFile4UTF8.xlsm	CSV形式ファイル読み出し機能サンプル(UTF-8版) ※マクロ有効ブック サンプルの起動プロシージャは「TEST2」です。(起動ボタン等はありません)
テストデータ.csv	下の画像の簡単なCSV形式ファイルサンプルデータ(シフトJIS版のみ)

サンプルマクロが入ったマクロ有効ブックが2つありますが、これはCSV形式ファイルの文字コードによるものでそれ以外に機能的な違いはありません。Excel 2016以降では「CSV UTF-8(コンマ区切り)(*.csv)」という保存形式が追加されているため、これに対応したものです。

同梱させているサンプルデータはシフトJIS版のみになるので、UTF-8にしたい場合はExcel 2016以降で開いて、「名前を付けて保存」で形式を変更させて下さい。

サンプルマクロは「TEST1」又は「TEST2」で、どちらもCSV形式ファイルを読み込んでから、内部で二次元配列を作成させてアクティブシートに貼り付ける動作になっています。CSV形式ファイルを直接開いたものと比較して動作が正しいか確認ができるようになっています。

この画面サンプルのように新規ブックを手前で開いて実行させると、マクロブック側にデータが貼り付くことが避けられます。



サンプルマクロを起動すると、開くCSV形式ファイルを指定するこの画面が表示されます。

CSV形式ファイルを指定して「開く」をクリックすると処理が実行されます。

添付のサンプルデータでは一瞬ですが、別の43列×43,000行のデータで試したところ、比較的新しいPCで5秒程度でした。

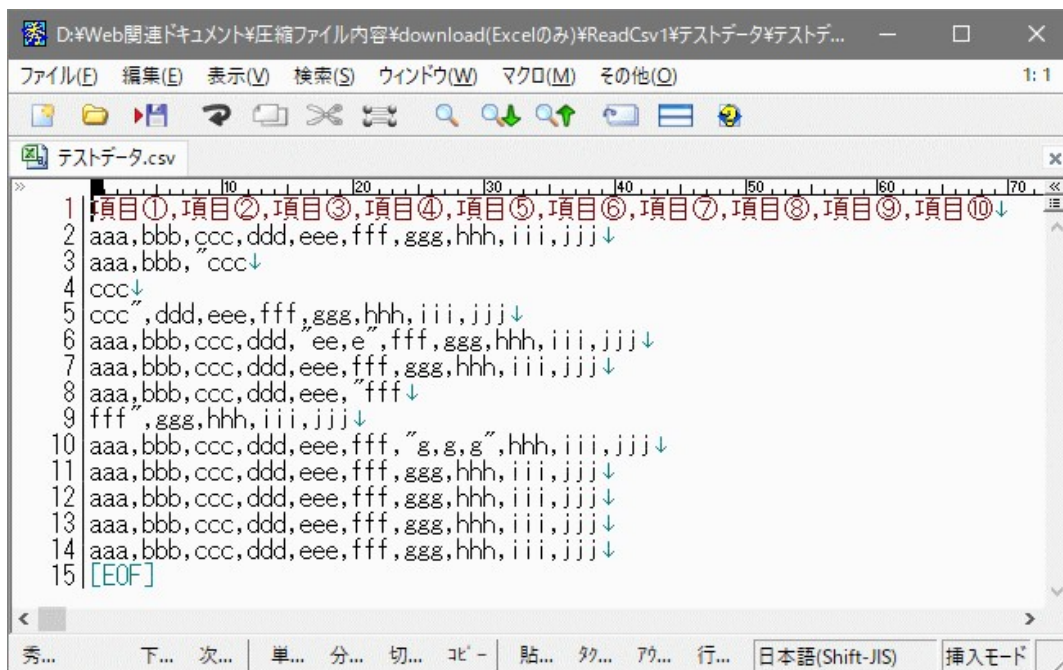
(ファイルサイズでは10MB程度)



これが処理結果のExcel画面です。

例えば、3行目はC列がセル内で改行されており、7行目のG列はセル内の文字列内にカンマが含まれているのが判ります。

Excel画面に展開するのならExcelで開けば良いのですが、これは検証用にやっていることであって本来の目的はExcelで開かずにマクロ上で必要な処理を行なうことです。



「CSV形式ファイル」というのは実体は「テキストファイル」です。「メモ帳」や「テキストエディタ」で開くことができます。行位置の説明をするため、行番号が表示される「テキストファイル」で開いてみました。1つ上のExcel画面と見比べて下さい。

まず、3～5行目はExcelシート上では3行目の内容であって、C列がセル内で2回セル内改行されているため、テキスト上では3行となっています。

8～9行目もF列がセル内改行されており、Excelシート上では6行目の内容になります。

「セル内改行」が「通常の改行」かの違いは、セル内改行があるセルは項目がダブルクォーテーションで囲われる規則となっており、ダブルクォーテーションで囲われた間での改行は「セル内改行」と判断して処理します。

実際の「セル内改行」はExcelシート上で「Alt+Enter」でキー入力された場合はLFコードのみなのですが、外部から読み込まれたり貼り付けられたりしたものはCR+LFコードになってしまうようです。

「セル内カンマ」についても同様でダブルクォーテーションで囲われた間でのカンマは「セル内カンマ」と判断して処理します。(G7セル、テキストは10行目) このことはサンプルにはありませんが「セル内ダブルクォーテーション」も同様で、ダブルクォーテーションの場合はセル値がダブルクォーテーションで囲われるのと同時に、セル内のダブルクォーテーションは2つ連記に変換されます。

他システムからのCSV形式ファイルを取り込む場合はこれらが前提条件となります。

もう一点ですが、このページは「カンマ数不定のCSVファイル読み込み」というタイトルですが、このサンプルマクロではModule1側の定数で行あたりの項目数を指定してチェックする方法を用意しています。

チェック自体を外してしまえば「カンマ数不定」にはなりますが、何かの処理に実装するとすればカンマ数は固定されるはずだし、ExcelからのCSV形式保存では行ごとにカンマ数がバラつくことはないはずです。

ということでソースコードです。まずは「Module1」です。

これは今回作成したクラスの動作確認のための起動部分であって、このままご利用いただくものではありません。

実際に実行する前に以下のモジュールレベル定数の値を確認して下さい。クラスを呼び出す時のプロパティ値になります。

定数名	意味
g_cnsCheckColCnt	カラム数によるチェックを行なうかどうかの判定です。Trueにすると不正カラム数のレコードがあるとエラーになります。Falseにした場合はチェックは行ないません。
g_cnsColCnt	チェックを行なう場合のカラム数です。 このサンプルの場合はこの定数をシートに貼り付ける列数として利用しています。実際のCSV形式ファイルの列数より大きい値になっていると実行時エラーになります。

```

' *****
'      CSV読み込みテスト                                     Module1 (Module)
'
'   作成者: 井上治   URL: http://www.ne.jp/asahi/excel/inoue/ [Excelでお仕事!]
' *****
' 変更日付 Rev  変更履歴内容----->
' 19/11/06 (1.00) 新規作成
' 19/11/09 (1.10) カラム数チェック有無の指定を追加
' *****
Option Explicit
'-----
Private Const g_cnsTitle As String = "CSV読み込みテスト"
Private Const g_cnsCheckColCnt As Boolean = True           ' カラム数チェック有無
Private Const g_cnsColCnt As Long = 10                    ' カラム数
' ↑ CSVファイルのカラム数をg_cnsColCntにセットしてから実行して下さい

```

```

'   チェック不要の場合はg_cnsCheckColCntをFalseに変更して下さい
'
' *****
' * 処理名   : TEST1
' * 機能     : CSV読み込みテスト
'
' -----
' * 戻り値   : (なし)
' * 引数     : (なし)
'
' -----
' * 作成日   : 2019年11月06日
' * 作成者   : 井上 治
' * 更新日   : 2019年11月09日
' * 更新者   : 井上 治
' * 機能説明 :
' * 注意事項 :
' *****
Public Sub TEST1()
'
'   テーブルINDEX (Row)
'   テーブルINDEX (Col)
'   テーブルINDEX 上限 (Row)
'   ファイル名
'   エラーメッセージ
'   ファイル名 (受け取り)
'   JAG配列テーブル
'   二次元配列テーブル
'
'   ファイル名受け取り
vntFilename = Application.GetOpenFilename("CSVファイル (*.csv;*.txt),*.csv;*.txt", , g_cnsTitle)
'   キャンセルは終了
If VarType(vntFilename) = vbBoolean Then Exit Sub
strFilename = vntFilename
'
'   CSV読み込みクラス (Ascii/シフトJIS)
With New clsReadCsvI
'   カラム数チェック有無を指定
.prpCheckColCnt = g_cnsCheckColCnt
'   カラム数を指定
.prpColCnt = g_cnsColCnt
'   CSV読み込み
If Not .ReadCsv(strFilename, tblRec) Then
MsgBox .prpErrMsg, vbCritical, g_cnsTitle
Exit Sub
End If
End With
'
'   以下はサンプルとしての結果検証用の記述です
'   カラム数チェックを行わない指定でもg_cnsColCntの指定列までしかシートには書き出されません
'
lnglRMax = UBound(tblRec)
ReDim tblRec2(lnglRMax, g_cnsColCnt - 1)
'   JAG配列テーブルを二次元配列テーブルに変換
Do While lnglR <= lnglRMax
lnglC = 0
'   カラム方向ループ
Do While lnglC < g_cnsColCnt
tblRec2(lnglR, lnglC) = tblRec(lnglR)(lnglC)
'   次へ
lnglC = lnglC + 1
Loop
'   次へ
lnglR = lnglR + 1
Loop
ReDim tblRec(0)
'
'   現在シートに貼り付け
With ActiveSheet
.Range(.Cells(1, 1), .Cells(lnglR, g_cnsColCnt)).Value = tblRec2
End With
End Sub
'
' -----<< End of Source >>-----

```

Module1では、どちらもほぼ同様のコーディングで中央付近の呼び出しクラス名が換わるだけです。

モジュール定数は必要に応じて変更して下さい。

指定ファイルの読み込みとファイル内容をJAG配列にるところまでを下で説明するクラス側が担当しており、それ以降の作業は戻されたJAG配列から処理本体が行なうので、このサンプルではJAG配列を二次元配列に置き換えてワークシートに貼り付けています。

念のために説明しておきますが「JAG配列」とは一次元配列の中に別の一次元配列を格納させる方法です。

VB系言語では、ReDimステートメントで配列要素数を動的に変更できるのは最終次元だけなので、Excelシートのような縦横の二次元配列の縦方向については途中で配列要素数の動的変更はできないのです。

ここで活躍するのが「JAG配列」です。「単純配列の中に単純配列を入れる」ということなので縦方向でも配列要素数の動的変更が可能です。

今回のようなCSV形式ファイルでは改行数をカウントできたとしても縦方向の配列要素数にはならないのですから、配列要

素数の動的変更は必須条件になります。

次は「clsReadCsv1」です。こちらはシフトJIS版の方の実処理クラスになります。UTF-8版の方は「clsReadCsv2」になっています。

公開プロシージャ「ReadCsv」は、先にプロパティ「prpCheckColCnt」でカラム数チェックの有無と、「prpColCnt」でCSV形式ファイルのカラム数を指定してから呼び出すようになっています。

```

' *****
'   CSV読み込みクラス                                     clsReadCsv1 (Class)
' *****
'   作成者: 井上 治   URL: http://www.ne.jp/asahi/excel/inoue/ [Excelでお仕事!]
' *****
'   [前提条件]
'   ・カンマ区切りテキストファイルであること
'   ・文字コードはAscii/シフトJIS形式
'   ・改行コードはCR、LF、CRLFに対応
'   ・フィールド数は全行一致していること(チェック有効時)
'   ・フィールド内で改行、あるいはフィールド内にカンマ、ダブルクォーテーションがある場合は
'     フィールド自体をダブルクォーテーションで囲うこと
' *****
'   [参照設定]
'   ・Microsoft Scripting Runtime
' *****
'   変更日付 Rev   変更履歴内容----->
'   19/11/06(1.00) 新規作成
'   19/11/09(1.10) LFコードのみの改行に対応、カラム数チェック有無の指定を追加
' *****
Option Explicit
' =====
Private Const g_cnsCom As String = ","
Private Const g_cnsDq As String = """"

Private g_blnCheckColCnt As Boolean           ' カラム数チェック有無
Private g_lngColCnt As Long                  ' チェック時のカラム数
Private g_strErrMsg As String                 ' エラーメッセージ

' *****
'   ■■■ 公開プロシージャ ■■■
' *****
'   * 処理名   : ReadCsv
'   * 機能     : CSV読み込み
' -----
'   * 戻り値   : 処理成否 (Boolean)
'   * 引数     : Arg1 = ファイル名 (String)           ※フルパス
'   *           : Arg2 = CSV内容JAG配列テーブル (Variant) ※Ref参照
' -----
'   * 作成日   : 2019年11月06日
'   * 作成者   : 井上 治
'   * 更新日   : 2019年11月09日
'   * 更新者   : 井上 治
'   * 機能説明 :
'   * 注意事項 :
' *****
Friend Function ReadCsv(ByVal strFilename As String, _
                        ByRef tblRec As Variant) As Boolean
' -----
    Dim strAllRec As String           ' レコード全体
    ReadCsv = False
    g_strErrMsg = ""
    ReDim tblRec(0)
    ' カラム数未設定は終了(チェック有効時)
    If g_blnCheckColCnt And g_lngColCnt = 0 Then
        g_strErrMsg = "カラム数が設定されていません。"
        Exit Function
    End If
    ' CSVファイル読み込み
    If Not FP_ReadFile(strFilename, strAllRec) Then Exit Function
    ' CSVファイル内容をJAG配列に変換
    If Not FP_GetJagTable(strAllRec, tblRec) Then Exit Function
    ' 終了
    ReadCsv = True
End Function

' *****
'   ■■■ サブ処理 ■■■
' *****
'   * 処理名   : FP_ReadFile
'   * 機能     : CSVファイル読み込み
' -----
'   * 戻り値   : 処理成否 (Boolean)
'   * 引数     : Arg1 = ファイル名 (String)           ※フルパス
'   *           : Arg2 = レコード全体 (String)       ※Ref参照
' -----
'   * 作成日   : 2019年11月06日
'   * 作成者   : 井上 治
'   * 更新日   : 2019年11月06日
'   * 更新者   : 井上 治

```

```

' * 機能説明 :
' * 注意事項 :
' *****
Private Function FP_ReadFile(ByVal strFilename As String, _
                             ByRef strAllRec As String) As Boolean
' -----
Dim objFso As FileSystemObject           ' FileSystemObject
Dim objTs As TextStream                 ' TextStream
Dim blnOpen As Boolean                  ' OPEN判定
FP_ReadFile = False
On Error GoTo ReadFile_ERROR
Set objFso = New FileSystemObject
' CSVファイルOPEN
Set objTs = objFso.OpenTextFile(strFilename, ForReading, False)
blnOpen = True
' ファイル全量を読み込み
strAllRec = objTs.ReadAll
FP_ReadFile = True
GoTo ReadFile_EXIT

' =====
ReadFile_ERROR:
g_strErrMsg = Err.Description

' =====
ReadFile_EXIT:
' CSVファイルCLOSE
If blnOpen Then objTs.Close
Set objTs = Nothing
Set objFso = Nothing
On Error GoTo 0
End Function

' *****
' * 処理名   : FP_GetJagTable
' * 機能     : CSVファイル内容をJAG配列に変換
' -----
' * 戻り値   : 処理成否 (Boolean)
' * 引数     : Arg1 = レコード全体 (String)
' *           : Arg2 = CSV内容JAG配列テーブル (Variant)           ※Ref参照
' -----
' * 作成日   : 2019年11月06日
' * 作成者   : 井上 治
' * 更新日   : 2019年11月09日
' * 更新者   : 井上 治
' * 機能説明 :
' * 注意事項 :
' *****
Private Function FP_GetJagTable(ByRef strAllRec As String, _
                                ByRef tblRec As Variant) As Boolean
' -----
' 全体で使用する変数
Dim lngIxRec As Long           ' レコードINDEX
Dim lngPos As Long             ' 現在文字位置
Dim lngPosEnd As Long          ' 最終文字位置
Dim lngCntRec As Long          ' レコードカウンタ
' レコード単位で使用する変数
Dim lngIxFld As Long           ' フィールドINDEX
Dim lngCntFld As Long          ' フィールドカウンタ
Dim blnCrLf As Boolean          ' 改行判定
Dim blnOkRec As Boolean        ' 有効レコード判定
Dim tblFld() As String         ' フィールドテーブル
' フィールド単位で使用する変数
Dim lngPosS As Long            ' 項目開始位置
Dim lngPosE As Long            ' 項目終了位置
Dim lngPosS2 As Long           ' 項目開始位置 (DQ調整後)
Dim lngPosE2 As Long           ' 項目終了位置 (DQ調整後)
Dim blnInText As Boolean        ' 文字列項目内判定
Dim blnDq As Boolean            ' ダブルクォーテーション判定
Dim blnDq2 As Boolean           ' 文字列内DQ発生判定
Dim strText As String           ' 項目テキストWORK

FP_GetJagTable = False
lngPos = 1
lngIxRec = -1
lngPosEnd = Len(strAllRec)
' 内容無しはエラー
If lngPosEnd = 0 Then
g_strErrMsg = "ファイルに内容が書き込まれていません。"
Exit Function
End If

' 最終文字位置まで繰り返す
Do While lngPos <= lngPosEnd
' -----
' レコード単位[前]処理
' -----

blnCrLf = False
blnOkRec = False

```

```

lngIdxFld = -1
lngCntFld = 0
ReDim tblFld(0)
' =====
' レコード単位[主]処理(レコード内項目巡回)
' =====
Do While lngPos <= lngPosEnd
    ' =====
    ' フィールド単位[前]処理
    ' =====
    blnInText = False
    blnDq = False
    blnDq2 = False
    ' 項目先頭位置
    lngPosS = lngPos
    ' =====
    ' フィールド単位[主]処理
    ' =====
    Do While lngPos <= lngPosEnd
        ' 現在文字を判定
        Select Case Mid(strAllRec, lngPos, 1)
            Case vbCr
                ' CRコード
                ' 文字列項目内でなければレコード終了
                If Not blnInText Then
                    blnCrLf = True
                    Exit Do
                End If
            Case vbLf
                ' LFコード
                ' 文字列項目内でなければレコード終了
                If Not blnInText Then
                    blnCrLf = True
                    Exit Do
                End If
            Case g_cnsDq
                ' ダブルクォーテーション
                ' 文字列内DQ発生判定か
                If blnInText Then
                    blnDq2 = True
                End If
                blnInText = Not blnInText
                blnDq = True
            Case g_cnsCom
                ' カンマ
                ' 文字列項目内でなければ項目終了
                If Not blnInText Then Exit Do
        End Select
        ' 次の文字へ
        lngPos = lngPos + 1
    Loop
    ' =====
    ' フィールド単位[後]処理
    ' =====
    ' 項目終了位置(実際は次のカンマ位置)
    lngPosE = lngPos
    lngPosS2 = lngPosS
    lngPosE2 = lngPosE
    ' ダブルクォーテーションで囲われているか
    If blnDq Then
        ' 両端文字(ダブルクォーテーション)を除外
        lngPosS2 = lngPosS + 1
        lngPosE2 = lngPosE - 1
    End If
    lngCntFld = lngCntFld + 1
    ' フィールドテーブル要素を追加
    lngIdxFld = lngIdxFld + 1
    ReDim Preserve tblFld(lngIdxFld)
    ' 有効なフィールドか(ブランクでないか)
    If lngPosE2 > lngPosS2 Then
        blnOkRec = True
        ' 項目テキストの取り出し
        strText = Mid(strAllRec, lngPosS2, lngPosE2 - lngPosS2)
        ' 文字列内ダブルクォーテーション発生判定
        If blnDq2 Then
            ' ダブルクォーテーション連記を解除(完璧な方法ではない)
            strText = Replace(strText, """"", "")
        End If
        ' フィールドテーブルに格納
        tblFld(lngIdxFld) = strText
    End If
    ' カンマ、改行を除外するため+1
    lngPos = lngPosE + 1
    ' レコード終了判定
    If blnCrLf Then Exit Do
Loop
' =====
' レコード単位[後]処理
' =====
' Cr後のLfは除外
If Mid(strAllRec, lngPos, 1) = vbLf Then
    lngPos = lngPos + 1
End If

```



```
' 有効レコードならJAG配列テーブルに格納
If blnOkRec Then
    lngCntRec = lngCntRec + 1
    ' 項目数判定
    If g_blnCheckColCnt And lngCntFld <> g_lngColCnt Then
        Call GP_AppendMessage(CStr(lngCntRec) & "REC目、項目数不正(" & lngCntFld & ")")
    End If
    lngIdxRec = lngIdxRec + 1
    ReDim Preserve tblRec(lngIdxRec)
    tblRec(lngIdxRec) = tblFld
End If
Loop
' 有効内容無し
If lngCntRec = 0 Then
    g_strErrMsg = "有効な内容がありません。"
End If
' エラーがなければOK
FP_GetJagTable = g_strErrMsg = ""
End Function

' *****
' * 処理名      : GP_AppendMessage
' * 機能        : メッセージ累積
' *****
' * 戻り値      : (なし)
' * 引数        : Arg1 = 今回メッセージ(String)
' *****
' * 作成日      : 2019年11月06日
' * 作成者      : 井上 治
' * 更新日      : 2019年11月06日
' * 更新者      : 井上 治
' * 機能説明    : 改行を挟んでメッセージを累積する
' * 注意事項    :
' *****
Private Sub GP_AppendMessage(ByVal strAddMSG As String)
    If g_strErrMsg <> "" Then g_strErrMsg = g_strErrMsg & vbCrLf
    g_strErrMsg = g_strErrMsg & strAddMSG
End Sub

' *****
' * ■■■ プロパティ ■■■
' *****
' * カラム数チェック有無(Boolean)
' *****
Friend Property Let prpCheckColCnt(ByVal blnValue As Boolean)
    g_blnCheckColCnt = blnValue
End Property

' *****
' * チェック時のカラム数(Long)
' *****
Friend Property Let prpColCnt(ByVal lngValue As Long)
    g_lngColCnt = lngValue
End Property

' *****
' * エラーメッセージ(String)
' *****
Friend Property Get prpErrMsg() As String
    prpErrMsg = g_strErrMsg
End Property

' -----<< End of Source >>-----
```

外部から呼ばれるプロシージャ「ReadCsv」の内部動作は次の2工程に分かれています。

プロシージャ	処理内容
FP_ReadFile	指定ファイル名のCSV形式ファイルをEOFまで全量を読み出し、文字列変数で返す処理です。この処理は「clsReadCsv1」と「clsReadCsv2」で文字コードの違いから処理記述が異なります。
FP_GetJagTable	渡されたファイル全量の内容から1文字ずつ判定しながら列方向の配列を作成し、さらに行方向の配列に列方向の配列を納めていく作業を行なう処理で、結果としてJAG配列を返します。この処理は「clsReadCsv1」「clsReadCsv2」とも処理記述は全く同じです。全体ループ（EOFまで）、レコード単位ループ（セル外の改行発見まで）、フィールドループ（項目境界かセル外改行が見つかるまで）の3重ループ処理になっています。

このクラスは実務にご利用いただけると幸いです。



←ReadCsvFile4.zip
(50KB)

このサイトの管理人ですが...

現在、仕事を頂ける企業様を探しております。

と言っても作成請負ではありません。今までも、情報システム部の手に負えないエンドユーザー寄りの細かいシステム開発をたくさんがけてきましたので、VB.NETやExcel VBAでの開発要望がある企業様があればお手伝いさせていただきたいのです。

「エンドユーザー寄りの細かいシステム」と言うと、一般には着手段階で詳細な仕様がきっちり揃うことはほとんどありません。システムの実現には要件の整理から入る必要があり、当然ながら事前に費用を見積もることもできません。

開発会社が入るのなら派遣やSES契約で一定期間に人を入れるのですが、これまた要求スキルの人材が得られるかどうかの問題がある上、中間に会社が入ることのコストも発生します。

私は今現在、単なる個人なので、月額固定的な期間契約がいただければ通勤可能な勤務地であればお手伝いできると思います。

問合せ等は、本サイトの「[意見・質問](#)」からお願いします。